

李宏毅_ML_Lecture_22

tags: Hung-yi Lee NTU Machine Learning

課程撥放清單 (<https://www.youtube.com/channel/UC2ggjtuuWvxrHHHiaDH1dIQ/playlists>).

ML Lecture 22: Ensemble

課程連結 ([https://www.youtube.com/watch?](https://www.youtube.com/watch?v=tH9FH1DH5n0&list=PLJV_el3uVTsPy9oCRY30oBPNLCo89yu49&index=32)

[v=tH9FH1DH5n0&list=PLJV_el3uVTsPy9oCRY30oBPNLCo89yu49&index=32](https://www.youtube.com/watch?v=tH9FH1DH5n0&list=PLJV_el3uVTsPy9oCRY30oBPNLCo89yu49&index=32)).

Framework of Ensemble

Framework of Ensemble

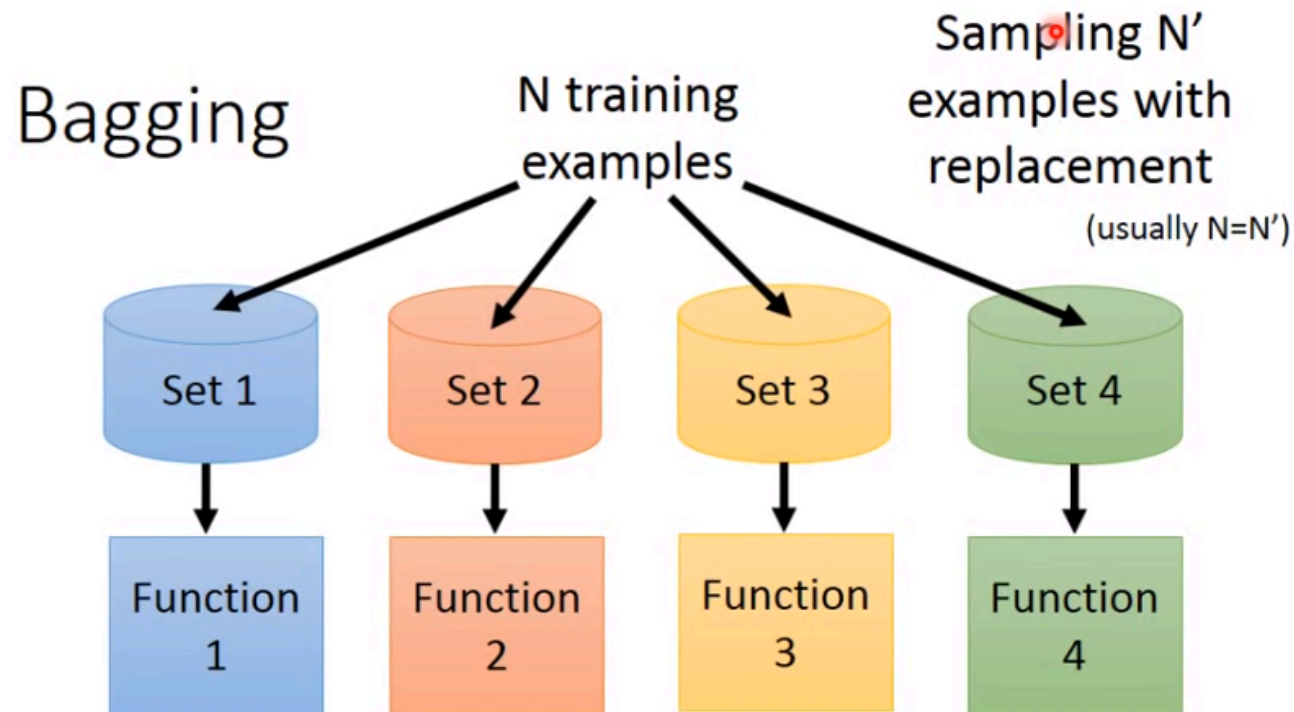
- Get a set of classifiers
 - $f_1(x), f_2(x), f_3(x), \dots$
 - 坦
 - 補
 - DD
 - Aggregate the classifiers (*properly*)
 - 在打王時每個人都有該站的位置
- They should be diverse.

Created with EverCam.
<http://www.camdemy.com>

Ensemble是一種團隊合作的概念，也許你手上有一堆的 classifier，而每一個 classifier 有不同的屬性，就好像打 game 一樣，每個人角色不同一起去圍歐 boss。

過去曾經說過，一個較小的模型可能會有較高的 Bias，較低的 Variance，而較大的模型會有較高的 Variance，較低的 Bias，如果我們可以將不同的模型組合起來，也許它的 Variance 就不會那麼大，將不同模型的輸出做平均來得到一個新的模型 $\hat{f}$ ，它所得的答案就可能跟真實是接近的。

Bagging



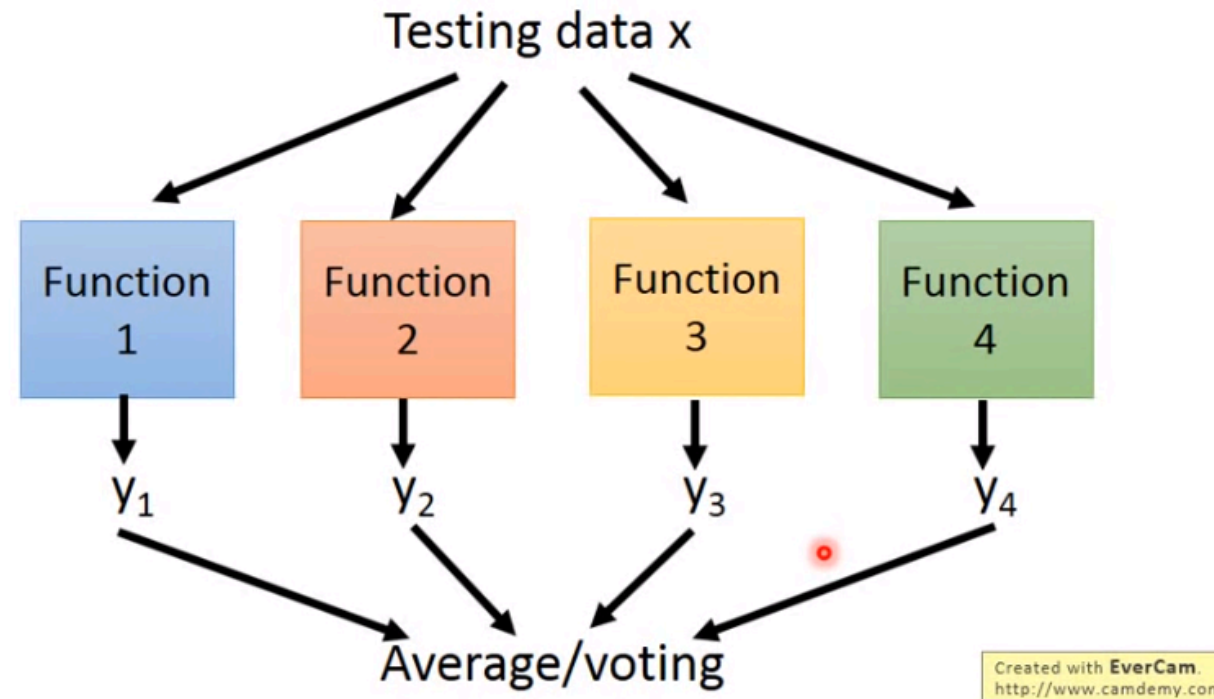
作法上如下：

1. 假設手上擁有 N 筆資料
2. 對資料集做採樣，每次取 N' 筆組成新的資料集
 - 採樣之後會再放回，因此如果 N' 設置為 N 所得結果不見得會與原資料集相同，因為還有放回(replacement)
3. 產生多個資料集，每個資料集內皆有 N' 筆資料
4. 訓練多個資料集，產生多個 function

註：課程說明為產生四個資料集，即產生四個 function

Bagging

Bagging This approach would be helpful when
your model is complex, easy to overfit.
e.g. decision tree



測試的時候將一筆測試資料丟到四個 function 中，將所得結果做
Average(regression)/Voting(classifier)，這時候得到的結果通常會比只有一個 function 的時候
Performance 較佳，即 Variance 較小，因此所得結果較為 robust。

問：什麼情況下我們需要做 Bagging?

答：當模型較為複雜，擔心結果過於 overfitting 的時候才會做 Bagging，執行 Bagging 是為了減低 Variance

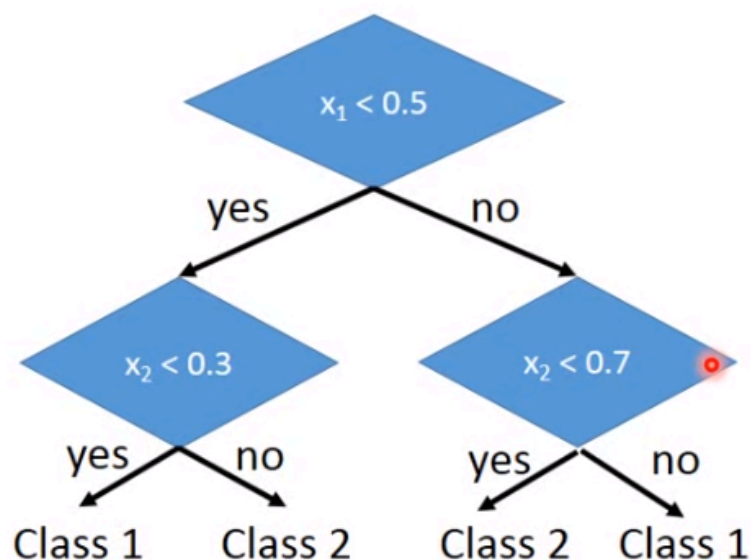
問：什麼模型容易 Overfitting?

答：如 Decision Tree，只要樹夠深就可以 100% 準

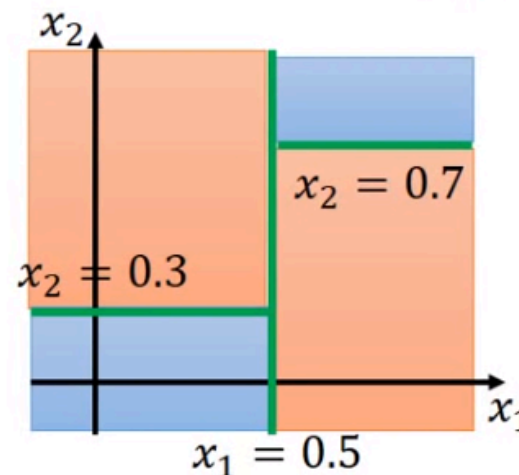
Decision Tree

Decision Tree

Assume each object x is represented by a 2-dim vector $\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$



Can have more complex questions



The questions in training

number of branches,
Branching criteria,
termination criteria,
base hypo

Created with EverCam.
<http://www.camdemy.com>

假設每一個 object 都有兩個 feature(x_1, x_2)，Decision Tree(決策樹)就是根據訓練資料來建置一棵樹。

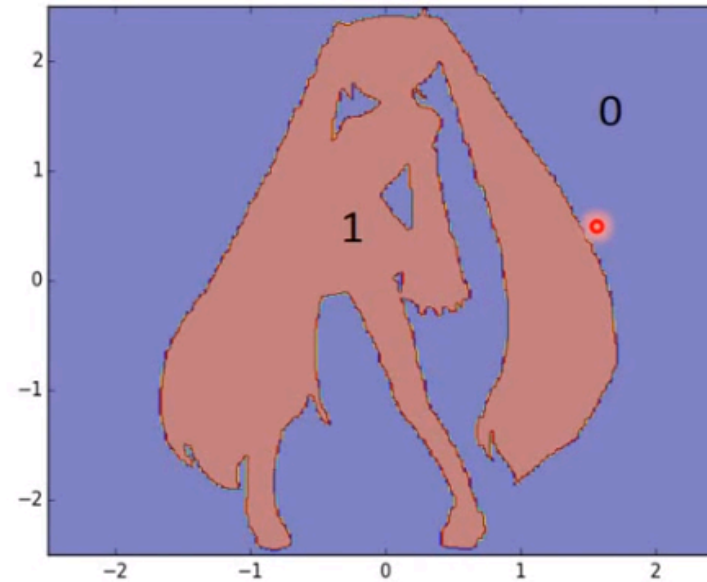
1. 首先條件 $x_1 < 0.5$ 就往左，反之往右

- 在 $x_1=0.5$ 上劃一直線區隔空間
- 2. $x_2 < 0.5$ 就類別 1(藍色) , 反之類別 2(橘色)
 - 在 $x_2=0.3$ 上劃一直線垂直第一條線 , 區隔空間
- 3. $x_2 < 0.7$ 就類別 2(橘色) , 反之類別 1(藍色)
 - 在 $x_2=0.7$ 上劃一直線垂直第一條線 , 區隔空間

這只是一個簡單的說明範例 , 實務上可以設計的更複雜 , 而且需要注意的問題很多。i.e. 每個節點多少分支、以什麼 Criterion(標準)做分支、什麼時候停止分支...

Experiment: Function of Miku

Experiment: Function of Miku



http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/theano/miku

(1st column: x, 2nd column: y, 3rd column: output (1 or 0))

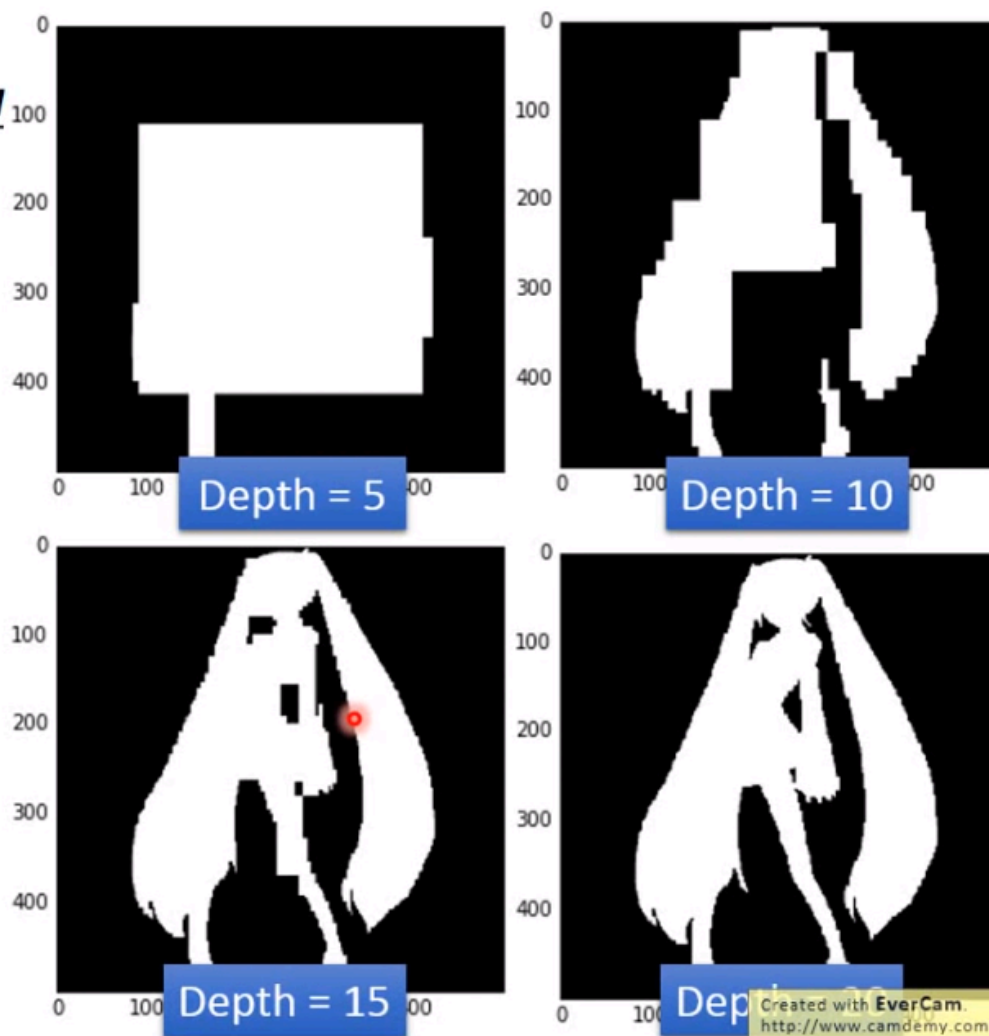
Created with EverCam.
<http://www.camdemy.com>

假設現在有一個資料分佈，狀似初音，初音內為 Class-1，初音外為 Class-0。

Experiment: Function of Miku

Experiment: Function of Miku

Single
Decision
Tree



可以發現到，當決策樹深度到達20的時候已經可以完美的區隔兩個類別，這並不令人意外，因為只要你想要，決策樹是永遠可以正確率100%。

Random Forest

Random Forest

train	f_1	f_2	f_3	f_4
x^1	O	X	O	X
x^2	O	X	X	O
x^3	X	O	O	X
x^4	X	O	X	O

- Decision tree:
 - Easy to achieve 0% error rate on training data
 - If each training example has its own leaf
- Random forest: Bagging of decision tree
 - Resampling training data is not sufficient
 - Randomly restrict the features/questions used in each split
- Out-of-bag validation for bagging

Created with EverCam.
<http://www.camdemy.com>

因為決策樹太容易 overfitting，因此有了 Random Forest(決策森林)，它是決策樹 Bagging 的一種實現。

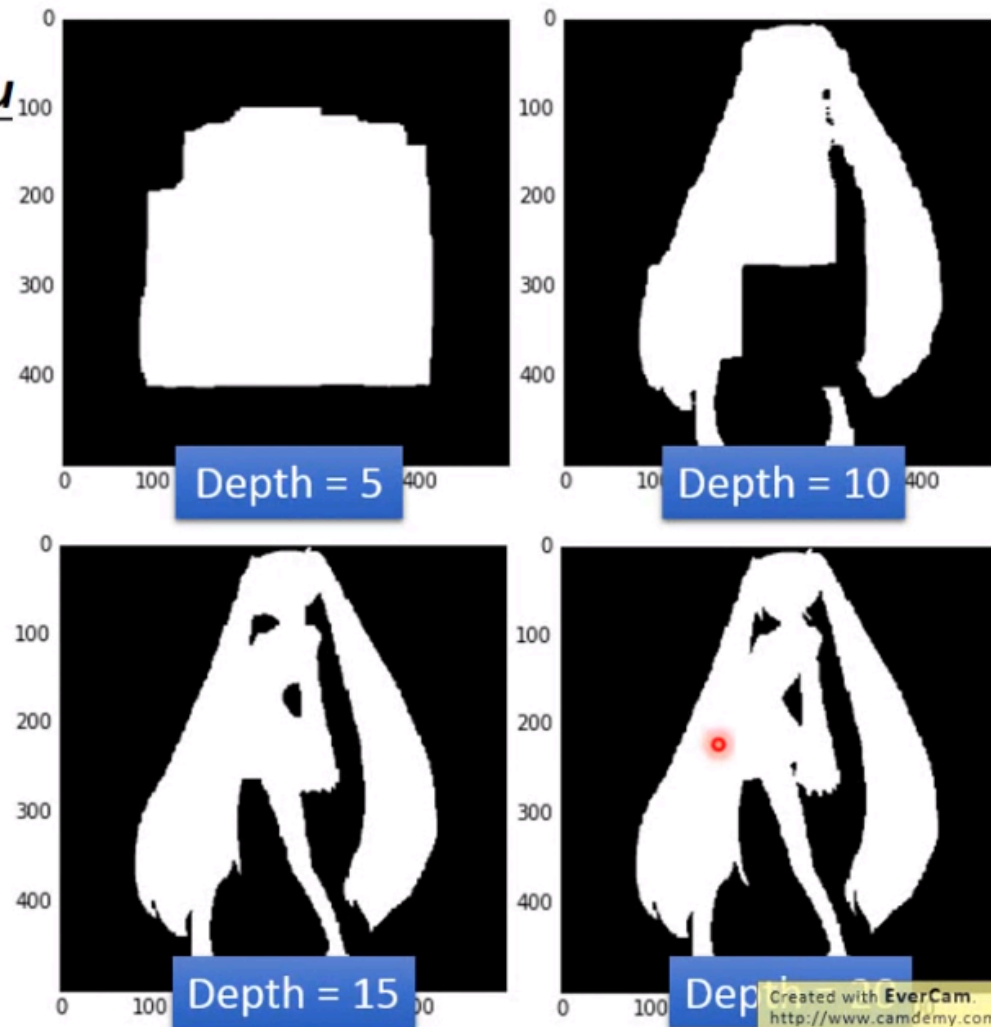
但如果使用稍早所說的採樣方法，那每所得的每一棵樹幾乎沒有差別，Random Forest的經典在於它每一次產生的tree都隨機的決定那些特徵或那些問題是不能用的，以此讓產生的tree都不相同，最後再做集合，得到最終的Random Forest。

Experiment: Function of Miku

Experiment: Function of Miku

Random
Forest

(100 trees)



我們發現到，當深度為 5 的時候得到的結果並不會差異過大，這是因為執行 Bagging 並不會使資料更擬合，我們得到的是將多棵樹的結果平均，較為平滑。

Boosting

Boosting

Training data:

$$\{(x^1, \hat{y}^1), \dots, (x^n, \hat{y}^n), \dots, (x^N, \hat{y}^N)\}$$

$\hat{y} = \pm 1$ (binary classification)

- Guarantee:
 - If your ML algorithm can produce classifier with error rate smaller than 50% on training data
 - You can obtain 0% error rate classifier after boosting.
- Framework of boosting
 - Obtain the first classifier $f_1(x)$
 - Find another function $f_2(x)$ to help $f_1(x)$
 - However, if $f_2(x)$ is similar to $f_1(x)$, it will not help a lot.
 - We want $f_2(x)$ to be complementary with $f_1(x)$ (How?)
 - Obtain the second classifier $f_2(x)$
 - Finally, combining all the classifiers
- The classifiers are learned sequentially.

Created with EverCam.
<http://www.camdemy.com>

Boosting 與 Bagging 的差異在於，Bagging 是應用於很強的 Model 上，而 Boosting 是應用於很弱的 Model 上。

Boosting's Guarantee(保證):

1. 如果你擁有一個機器學習演算法，並且在訓練資料集上有著 50% 以上的錯誤率
2. Boosting 可以保證將這些錯誤率略高於 50% 的模型組合起來之後給你一個錯誤率 0%

作法如下：

1. 尋找第一個 classifier $f_1(x)$
 - 很弱沒有關係
2. 找 classifier $f_2(x)$ 來協助 $f_1(x)$
 - 兩者之間是互補的關係，因此不能很像
3. 找出跟 $f_1(x)$ 最互補的 $f_2(x)$
4. 相同方式找出 $f_3(x)$...

找到一堆 classifier 之後，將它們集合起來，就可以得到很低的 error_rate，即使每一個 classifier 都很弱也可以辦的到。

也需要注意到，與 Bagging 不同的第二個點在於，Boosting 是有順序的執行，而 Bagging 可以無序。因此 Boosting 要先擁有 $f_1(x)$ ，再有 $f_2(x)$...，但 Bagging 如 Random Forest 可以亂數擁有。

How to obtain different classifiers?

How to obtain different classifiers?

- Training on different training data sets
- How to have different training data sets
 - Re-sampling your training data to form a new set
 - Re-weighting your training data to form a new set
 - In real implementation, you only have to change the cost/objective function

$$(x^1, \hat{y}^1, u^1) \quad u^1 = \cancel{1} \quad 0.4$$

$$(x^2, \hat{y}^2, u^2) \quad u^2 = \cancel{1} \quad 2.1$$

$$(x^3, \hat{y}^3, u^3) \quad u^3 = \cancel{1} \quad 0.7$$

$$L(f) = \sum_n l(f(x^n), \hat{y}^n)$$



$$L(f) = \sum_n \color{red}{u}^n l(f(x^n), \hat{y}^n)$$

Created with EverCam.
http://www.camdemy.com

類似於 Bagging 取得不同資料集的方式，在 Boosting 也可以利用 Re-sampling(重新抽樣)的方式來製造不同的訓練資料集，以此得到不同的 classifier。

或者可以利用權重的變更來產生不同的資料集，如上圖所示，原始資料集權重皆為1，透過調整 u 的方式來產生不同的資料集，在考慮 Loss Function 的時候就單純的在每筆資料集乘上它的權重即可。

Idea of Adaboost

Idea of Adaboost

- Idea: **training $f_2(x)$ on the new training set that fails $f_1(x)$**
- How to find a new training set that fails $f_1(x)$?

ε_1 : the error rate of $f_1(x)$ on its training data

$$\varepsilon_1 = \frac{\sum_n u_1^n \delta(f_1(x^n) \neq \hat{y}^n)}{Z_1} \quad Z_1 = \sum_n u_1^n \quad \varepsilon_1 < 0.5$$

Changing the example weights from u_1^n to u_2^n such that

$$\frac{\sum_n u_2^n \delta(f_1(x^n) \neq \hat{y}^n)}{Z_2} = 0.5$$

The performance of f_1 for new weights would be random.

Training $f_2(x)$ based on the new weights u_2^n

Boosting的方式有很多，最為經典的即是『Adaboost』。

直觀說明如下：

1. 訓練一個 classifier f_1

- ϵ_1 _error-rate，加總計算每一筆訓練資料的結果是否正確，正確為0，錯誤為1，並且乘上權重 u^n (每一筆資料各自權重)，再做 Normalization (除上 Z_1)
 - Z_1 即為所有資料權重加總
- ϵ_1 一定小於0.5

2. re-weight 一組新的訓練資料集讓 f_1 效能爛掉

- 將權重調整變更為 u_2 ，讓 error-rate 變為0.5

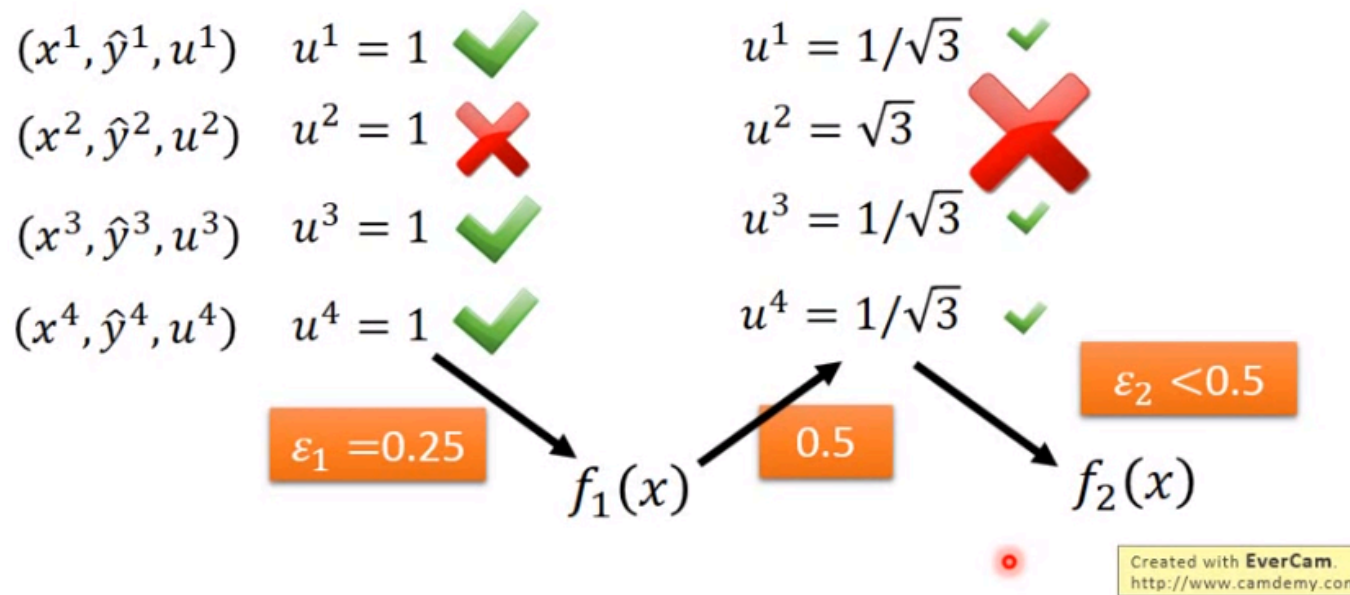
3. 利用 re-weight 的訓練資料集訓練 f_2

- 利用 u_2 產生的新資料集訓練 f_2

Re-Weighting Training Data

Re-weighting Training Data

- Idea: **training $f_2(x)$ on the new training set that fails $f_1(x)$**
- How to find a new training set that fails $f_1(x)$?



舉例來說，目前有四筆訓練資料集，各自權重為 $u^1 \dots u^4$ ，並且預設值皆為 1。

1. 訓練得到 f_1 ，得到 error-rate 為 $0.25(\epsilon_1)$
2. 調整權重，讓 f_1 的 error-rate 變為 0.5

- 答對的權重減小，答錯的權重加大

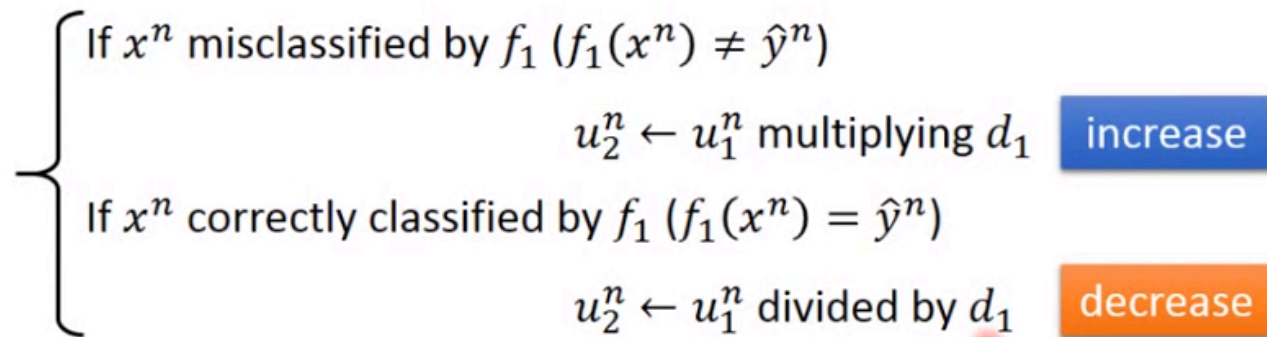
3. 利用調整權重的資料訓練 f_2 ，得到 ϵ_2 小於 0.5

後續會有更完整的案例說明。

Re-Weighting Training Data

Re-weighting Training Data

- Idea: **training $f_2(x)$ on the new training set that fails $f_1(x)$**
- How to find a new training set that fails $f_1(x)$?



f_2 will be learned based on example weights u_2^n

What is the value of d_1 ?

Re-Weight 的作法如下：

1. 當 x^n 分類錯誤即 u_1^n 乘上 d_1 得 u_2^n
 - 加大
2. 當 x^n 分類正確即 u_1^n 除 d_1 得 u_2^n
 - 縮小

而 d_1 的功能就是讓 u_1 變為 u_2 之後使得 f_1 的 error rate 變為 0.5

Re-Weighting Training Data

Re-weighting Training Data

$$\varepsilon_1 = \frac{\sum_n u_1^n \delta(f_1(x^n) \neq \hat{y}^n)}{Z_1} \quad Z_1 = \sum_n u_1^n$$

$$\frac{\sum_n u_2^n \delta(f_1(x^n) \neq \hat{y}^n)}{Z_2} = 0.5 \quad \begin{array}{l} f_1(x^n) \neq \hat{y}^n \quad u_2^n \leftarrow u_1^n \text{ multiplying } d_1 \\ f_1(x^n) = \hat{y}^n \quad u_2^n \leftarrow u_1^n \text{ divided by } d_1 \end{array}$$

$$= \sum_{f_1(x^n) \neq \hat{y}^n} u_1^n d_1 = \sum_{f_1(x^n) \neq \hat{y}^n} u_2^n + \sum_{f_1(x^n) = \hat{y}^n} u_2^n$$

$$= \sum_n u_2^n = \sum_{f_1(x^n) \neq \hat{y}^n} u_1^n d_1 + \sum_{f_1(x^n) = \hat{y}^n} u_1^n / d_1$$

$$\frac{\sum_{f_1(x^n) \neq \hat{y}^n} u_1^n d_1 + \sum_{f_1(x^n) = \hat{y}^n} u_1^n / d_1}{\sum_{f_1(x^n) \neq \hat{y}^n} u_1^n d_1} = 2$$

Created with EverCam.
http://www.camdemy.com

分子的部份是總計分類錯誤的資料(紅箭頭所指之處)， u_2 是由 $d_1 u_1$ 所得，因此可以寫成 $\sum u_1^n d_1$ 。

分母的部份是總計 u_2 ，它包含了正確與錯誤的資料，因此可得到 $\sum u_2 = \sum u_1 d_1 + \sum u_1 / d_1$

將式子帶入並翻轉，得到最下面的數學式

Re-Weighting Training Data

Re-weighting Training Data

$$\varepsilon_1 = \frac{\sum_n u_1^n \delta(f_1(x^n) \neq \hat{y}^n)}{Z_1} \quad Z_1 = \sum_n u_1^n$$

$$\frac{\sum_n u_2^n \delta(f_1(x^n) \neq \hat{y}^n)}{Z_2} = 0.5 \quad \begin{array}{l} f_1(x^n) \neq \hat{y}^n \quad u_2^n \leftarrow u_1^n \text{ multiplying } d_1 \\ f_1(x^n) = \hat{y}^n \quad u_2^n \leftarrow u_1^n \text{ divided by } d_1 \end{array}$$

$$\frac{\sum_{f_1(x^n) \neq \hat{y}^n} u_1^n d_1 + \sum_{f_1(x^n) = \hat{y}^n} u_1^n / d_1}{\sum_{f_1(x^n) \neq \hat{y}^n} u_1^n d_1} = 2 \quad \frac{\sum_{f_1(x^n) = \hat{y}^n} u_1^n / d_1}{\sum_{f_1(x^n) \neq \hat{y}^n} u_1^n d_1} = 1$$

$$\sum_{f_1(x^n) = \hat{y}^n} u_1^n / d_1 = \sum_{f_1(x^n) \neq \hat{y}^n} u_1^n d_1 \quad \frac{1}{d_1} \sum_{f_1(x^n) = \hat{y}^n} u_1^n = d_1 \sum_{f_1(x^n) \neq \hat{y}^n} u_1^n$$

$$\varepsilon_1 = \frac{\sum_{f_1(x^n) \neq \hat{y}^n} u_1^n}{Z_1} \quad \frac{Z_1(1 - \varepsilon_1)}{d_1} = Z_1 \varepsilon_1 d_1$$

$$\sum_{f_1(x^n) \neq \hat{y}^n} u_1^n = Z_1 \varepsilon_1 \quad d_1 = \sqrt{(1 - \varepsilon_1) / \varepsilon_1} > 1$$

數學式來看，分子前項與分母相同，因此可得分子後項除分母會得到1，再將分母移至等號右邊...最後我們得到一個結論， $d_1 = \sqrt{(1 - \varepsilon_1) / \varepsilon_1}$

再拿這個 d_1 來產生新的資料集就可以讓 f_1 的 error rate 變更為 0.5，要注意到， d_1 一定大於 1，這是因為 ϵ_1 小於 0.5，根號內的項目分子會大於分母所造成。

Algorithm for AdaBoost

Algorithm for AdaBoost

- Giving training data $\{(x^1, \hat{y}^1, u_1^1), \dots, (x^n, \hat{y}^n, u_1^n), \dots, (x^N, \hat{y}^N, u_1^N)\}$
 - $\hat{y} = \pm 1$ (Binary classification), $u_1^n = 1$ (equal weights)
- For $t = 1, \dots, T$:
 - Training weak classifier $f_t(x)$ with weights $\{u_t^1, \dots, u_t^N\}$
 - ϵ_t is the error rate of $f_t(x)$ with weights $\{u_t^1, \dots, u_t^N\}$
 - For $n = 1, \dots, N$:
 - If x^n is misclassified by $f_t(x)$: $\hat{y}^n \neq f_t(x^n)$
 - $u_{t+1}^n = u_t^n \times d_t = u_t^n \times \exp(\alpha_t)$ $d_t = \sqrt{(1 - \epsilon_t)/\epsilon_t}$
 - Else:
 - $u_{t+1}^n = u_t^n / d_t = u_t^n \times \exp(-\alpha_t)$ $\alpha_t = \ln \sqrt{(1 - \epsilon_t)/\epsilon_t}$

$$u_{t+1}^n \leftarrow u_t^n \times \exp(-\hat{y}^n f_t(x^n) \alpha_t)$$

Created with EverCam.
http://www.camdemy.com

AdaBoost 說明如下：

1. 我們手上有一個資料集共計 N 筆，預設權重為 1 ，即 $u_1^n = 1$ ，分類正確為 1 ，錯誤為 -1
2. 執行 T 次的迭代，每次的迭代都得到一個 f_t ，集合起來就是一個超強的分類器
 - 每次的迭代 t 都有自己的權重 u_t^n
 - 利用 u_t^n 計算第 t 次迭代的 ϵ_t
 - Re-Weight
 - 分類錯誤： $u_{t+1}^n = u_t^n * d_t$
 - 分類正確： $u_{t+1}^n = u_t^n / d_t$
 - $d_t = \sqrt{(1 - \epsilon_t) / \epsilon_t}$

也可以將 d_t 取 $\ln$ ，即 $\alpha_t = \ln \sqrt{(1 - \epsilon_t) / \epsilon_t}$ ，計算數學式也會連動變更，這麼做的好處是可以讓表達式更簡便，以一個式子表示即可，如下：

$$u_{t+1}^n \leftarrow u_t^n \times \exp(-\hat{y}^n f_t(x^n) \alpha_t)$$

當分類正確，得到正 1 ，正負得負，就進入分類正確的數學式，反正分類錯誤得到負 1 ，負負得正，就進入分類錯誤的數學式。

Algorithm for AdaBoost

Algorithm for AdaBoost

- We obtain a set of functions: $f_1(x), \dots, f_t(x), \dots, f_T(x)$
- How to aggregate them?

- Uniform weight:

- $H(x) = \text{sign}(\sum_{t=1}^T f_t(x))$

- Non-uniform weight:

- $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t f_t(x))$

Smaller error ε_t ,
larger weight for
final voting

$$\alpha_t = \ln \sqrt{(1 - \varepsilon_t) / \varepsilon_t}$$

$$\varepsilon_t = 0.1$$

$$\varepsilon_t = 0.4$$

$$u_{t+1}^n = u_t^n \times \exp(-\hat{y}^n f_t(x^n) \alpha_t)$$

$$\alpha_t = 1.10$$

$$\alpha_t = 0.20$$

Created with EverCam.
<http://www.camdemy.com>

現在我們的手上有了一把手的分類器，是時候來結合它們：

1. Uniform weight

- 將所有的 Output 加總看得到的是正值或是負值

2. Non-uniform weight

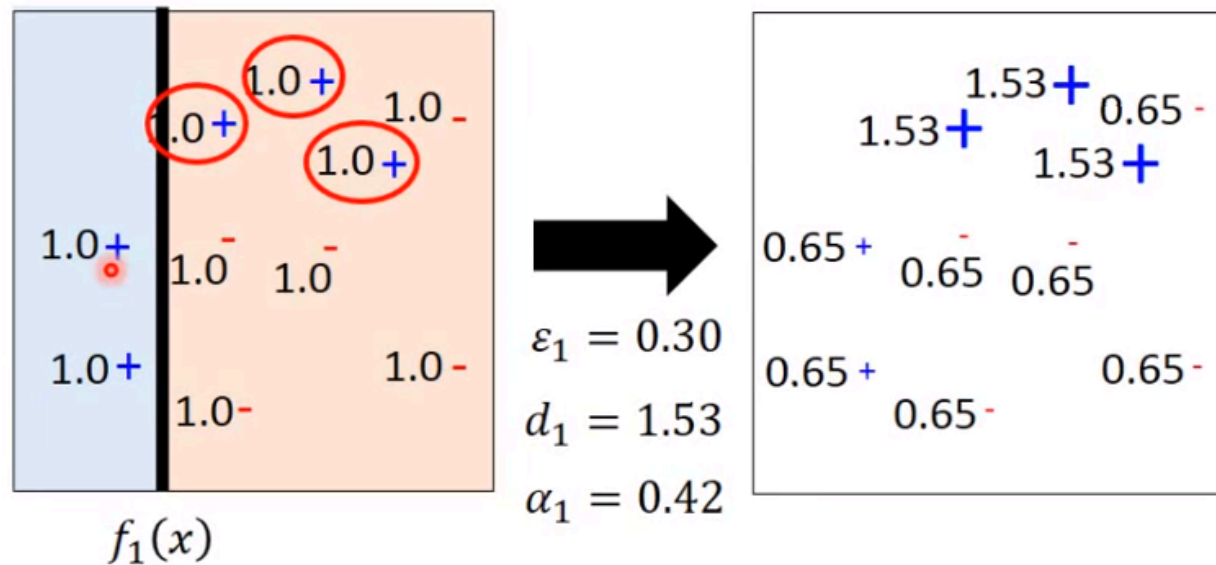
- 上述用法(Uniform weight)雖然可行，但不是最好的，因為每一個分類器有效能有好有壞，因此要利用權重調整。
- 在每一個分類器的 Output 都乘上一個權重 α_t ，再加總取正負號。
- α_t 即是先前所提過改變資料權重的數值
 - $\alpha_t = \ln \sqrt{(1 - \epsilon_t) / \epsilon_t}$
 - i.e. $\epsilon_t = 0.1$ 帶入計算即得 $\alpha_t = 1.10$
 - 錯誤率較低的分類器會得到較高的權重 α_t ，另一例的 $\epsilon_t = 0.4$ ，所得權重僅0.2。

Toy Example

Toy Example

$T=3$, weak classifier = decision stump

• $t=1$



Created with EverCam.
<http://www.camdemy.com>

舉例說明 · $T = 3$ · $n = 10$ · weak classifier = decision stump(見下附註說明)

• $t=1$

◦ 初始權重皆為1

- 找出 $f_1(x)$ ，並有三筆資料錯誤
- $\epsilon_1 = 0.3(3/10)$
- $d_1 = 0.53$
- $\alpha_1 = 0.42$
- 改變權重(正確減小，錯誤加大)
 - 正確：除 1.53
 - 錯誤：乘 1.53

註 1：Decision Stump: 假設 feature 分佈於二維平面上，在平面上選擇一個 Dimension 切一刀，一邊是 class-1，一邊是 class-2

註 2：實作 Boosting 一定要選擇弱分類器(weak classifier)，因此選擇 Decision Stump

註 3： α_t 為每個分類器的權重

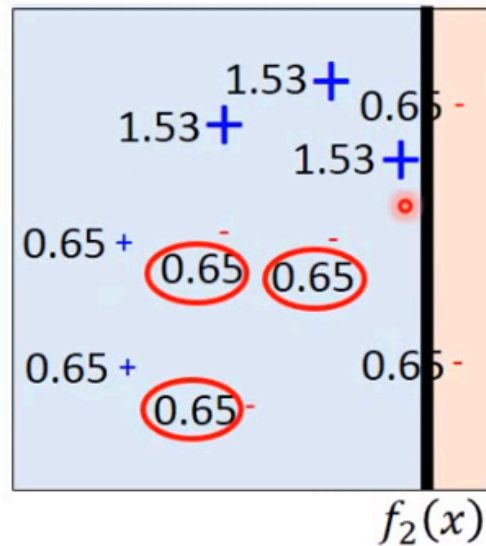
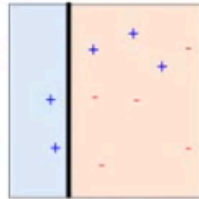
Toy Example

Toy Example

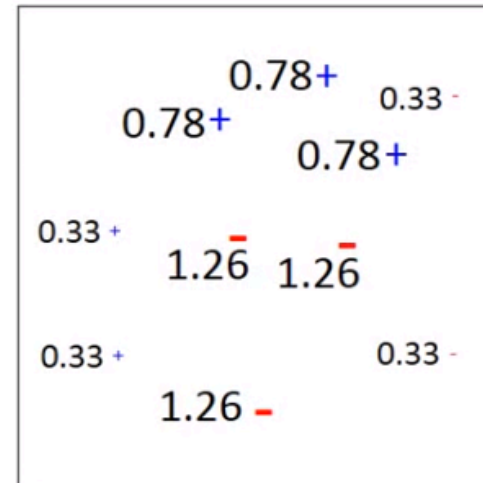
$T=3$, weak classifier = decision stump

• $t=2$

$f_1(x)$:
 $\alpha_1 = 0.42$



$\epsilon_2 = 0.21$
 $d_2 = 1.94$
 $\alpha_2 = 0.66$



Created with EverCam.
<http://www.camdemy.com>

尋找第二個 Decision stump

- $t=2$
 - 權重依分類結果調整

- 利用新的權重找出 $f_2(x)$ ，並有三筆資料錯誤
- $\epsilon_2 = 0.21$
- $d_2 = 1.94$
- $\alpha_2 = 0.66$
- 改變權重(正確減小，錯誤加大)
 - 正確：除 1.94
 - 錯誤：乘 1.94

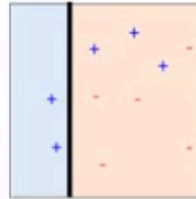
Toy Example

Toy Example

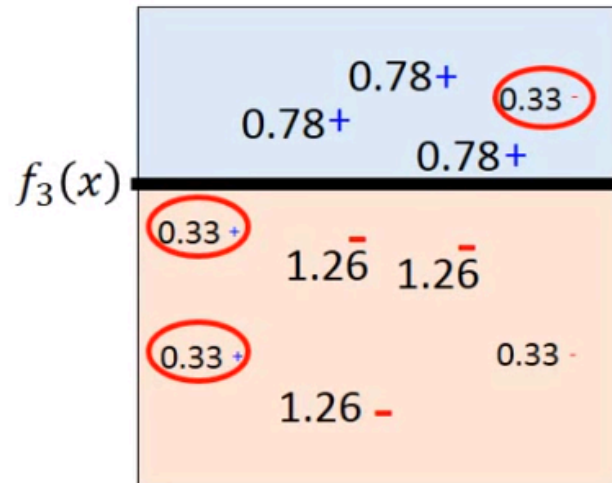
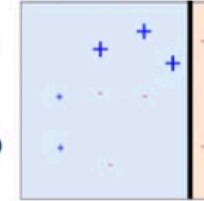
$T=3$, weak classifier = decision stump

• $t=3$

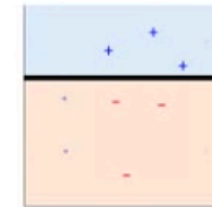
$f_1(x)$:
 $\alpha_1 = 0.42$



$f_2(x)$:
 $\alpha_2 = 0.66$



$f_3(x)$:
 $\alpha_3 = 0.95$



$\epsilon_3 = 0.13$

$d_3 = 2.59$

$\alpha_3 = 0.95$

Created with EverCam.
<http://www.camdemy.com>

尋找第三個 Decision stump

• $t=3$

◦ 權重依分類結果調整

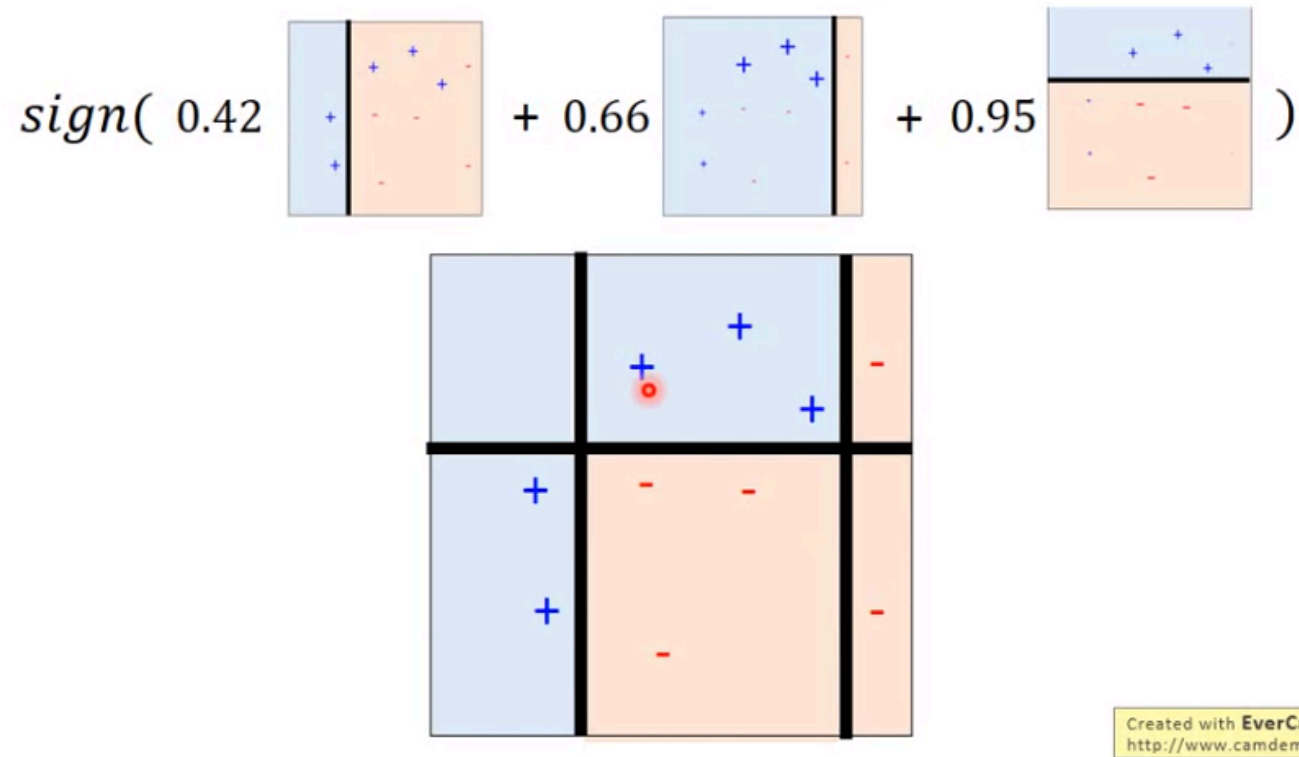
- 利用新的權重找出 $f_3(x)$ ，並有三筆資料錯誤
- $\epsilon_3 = 0.13$
- $d_3 = 2.59$
- $\alpha_3 = 0.95$

因為我們預設執行 $T=3$ ，即 3 個分類器，因此只到這邊就結束了。

Toy Example

Toy Example

- Final Classifier: $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t f_t(x))$



將每一個 Classifier 的 Output 乘上對應的權重 α_t 再取正負號來決定最後的 Output 結果。

上圖下將決策邊界區隔為六個空間說明：

- 上左：三個分類器皆同意為藍
- 上中： f_1 覺得是紅，但另兩個權重加總較大，因此為藍
- 上右： f_3 覺得是藍，但另兩個權重加總較大，因此為紅
- 下左： f_3 覺得是紅，但另兩個權重加總較大，因此為藍
- 下中： f_2 覺得是藍，但另兩個權重加總較大，因此為紅
- 下右：三個分類器皆同意為紅

很奇妙的是，三個分類器皆有錯誤，沒有一個是完美的，但是將這三個 Decision Stump 組合起來之後卻得到一個正確率 100% 的結果。

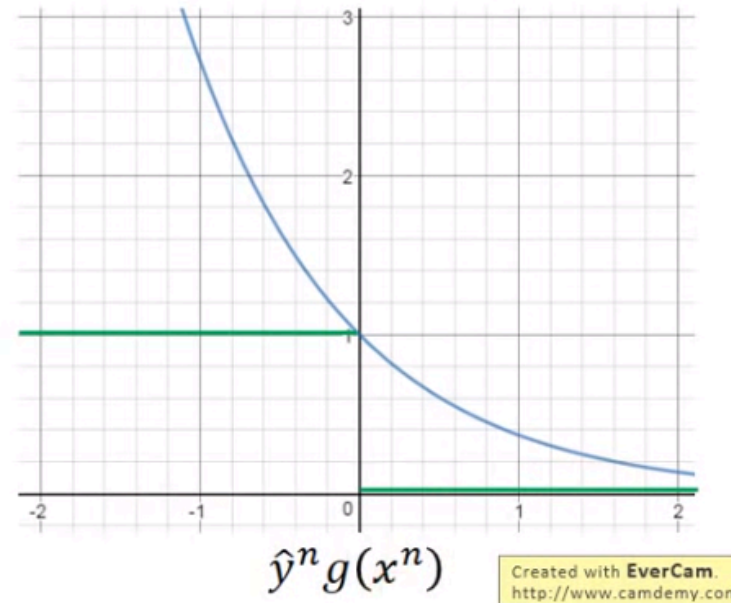
Error Rate of Final Classifier

Error Rate of Final Classifier

- Final classifier: $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t f_t(x))$
 $\alpha_t = \ln \sqrt{(1 - \varepsilon_t) / \varepsilon_t}$ $g(x)$

Training Data Error Rate

$$\begin{aligned}
 &= \frac{1}{N} \sum_n \delta(H(x^n) \neq \hat{y}^n) \\
 &= \frac{1}{N} \sum_n \delta(\hat{y}^n g(x^n) < 0) \\
 &\leq \frac{1}{N} \sum_n \exp(-\hat{y}^n g(x^n))
 \end{aligned}$$



之前的課程說明了 Boosting 的精神，接著要證明，證明當分類器愈多(或 Adaboost 執行的 iteration 愈多)在訓練資料集上的失誤會愈小。

1. 計算 $H(x)$ 的 error rate

- 即加總錯誤筆數(n)除總資料數(N)

2. 以 $g(x)$ 替代數學式

- 代表 T 個分類器的權重加總
- 同號 error 為 0，異號為 1

注意到 error rate 是存在 upper bound，如圖所示，藍線區即為 upper bound。

Training Data Error Rate

Training Data Error Rate

$$\leq \frac{1}{N} \sum_n \exp(-\hat{y}^n g(x^n))$$

$$g(x) = \sum_{t=1}^T \alpha_t f_t(x)$$

$$\alpha_t = \ln \sqrt{(1 - \varepsilon_t) / \varepsilon_t}$$

Z_t : the summation of the weights of training data when training f_t

What is Z_{T+1} =? $Z_{T+1} = \sum_n u_{T+1}^n$

$$\left. \begin{array}{l} u_1^n = 1 \\ u_{t+1}^n = u_t^n \times \exp(-\hat{y}^n f_t(x^n) \alpha_t) \end{array} \right\} u_{T+1}^n = \prod_{t=1}^T \exp(-\hat{y}^n f_t(x^n) \alpha_t)$$

$$\begin{aligned} Z_{T+1} &= \sum_n \prod_{t=1}^T \exp(-\hat{y}^n f_t(x^n) \alpha_t) \\ &= \sum_n \exp\left(-\hat{y}^n \sum_{t=1}^T f_t(x^n) \alpha_t\right) \end{aligned}$$

Created with EverCam.
http://www.camdemy.com

我們要證明的是 Upper Bound 會愈來愈小，在證明之前先計算另一個數值 Z_{T+1} (即第 $T+1$ 次迭代的加總值)

我們知道，初始權重 u_1^n 為 1，後續再依所得的 ϵ 來計算第二次、第三次...第 T 次的權重值，即 $u_{t+1}^n = u_t^n \times \exp(-\hat{y}^n f_t(x^n) \alpha_t)$ ，直觀來看就是每一筆資料都會乘上 T 次的 exponential 項目所得值。

因此得到如下推導：

1. $Z_{T+1} = \sum_n \prod_{t=1}^T \exp(-\hat{y}^n f_t(x^n) \alpha_t)$
2. $= \sum_n \exp(-\hat{y}^n \sum_{t=1}^T f_t(x^n) \alpha_t)$
 - 將連乘 $\prod$ 放至 exponential 內，乘項放至指數會是相加
 - 其中 $\hat{y}$ 是實際的 label，與 iteration 無關
 - $\sum_{t=1}^T f_t(x^n) \alpha_t$ 即是 $g(x)$ ，
3. $= \sum_n \exp(-\hat{y}^n g(x))$
 - 此項目即為 Upper Bound

上面推導之後我們發現到，Error 的 Upper Bound 就是 $\frac{1}{N} Z_{T+1}$ ，這代表著訓練資料集中的權重加總與 Error Upper Bound 是相關的。

因此，接下來我們只要證明 Weight 的 Summation 愈來愈小就可以。可以想像的是，當分類正確的時候權重是愈小，因此加總權重值愈小的時候就代表愈見正確

Z_t ：加總每一筆訓練資料集的權重值

Training Data Error Rate

Training Data Error Rate

$$\leq \frac{1}{N} \sum_n \exp(-\hat{y}^n g(x^n)) = \frac{1}{N} Z_{T+1}$$

$$g(x) = \sum_{t=1}^T \alpha_t f_t(x)$$

$$\alpha_t = \ln \sqrt{(1 - \epsilon_t) / \epsilon_t}$$

$$Z_1 = N \quad (\text{equal weights})$$

$$Z_t = \underline{Z_{t-1} \epsilon_t} \exp(\alpha_t) + \underline{Z_{t-1} (1 - \epsilon_t)} \exp(-\alpha_t)$$

Misclassified portion in Z_{t-1} Correctly classified portion in Z_{t-1}

$$= Z_{t-1} \epsilon_t \sqrt{(1 - \epsilon_t) / \epsilon_t} + Z_{t-1} (1 - \epsilon_t) \sqrt{\epsilon_t / (1 - \epsilon_t)}$$

$$= Z_{t-1} \times 2\sqrt{\epsilon_t (1 - \epsilon_t)} \quad Z_{T+1} = N \prod_{t=1}^T 2\sqrt{\epsilon_t (1 - \epsilon_t)}$$

$$\text{Training Data Error Rate} \leq \prod_{t=1}^T \underline{2\sqrt{\epsilon_t (1 - \epsilon_t)}} < 1$$

Smaller and smaller

Created with EverCam.
http://www.camdemy.com

上面看到的是 Z_{t-1} 的計算推導，最終得到的數學式如下：

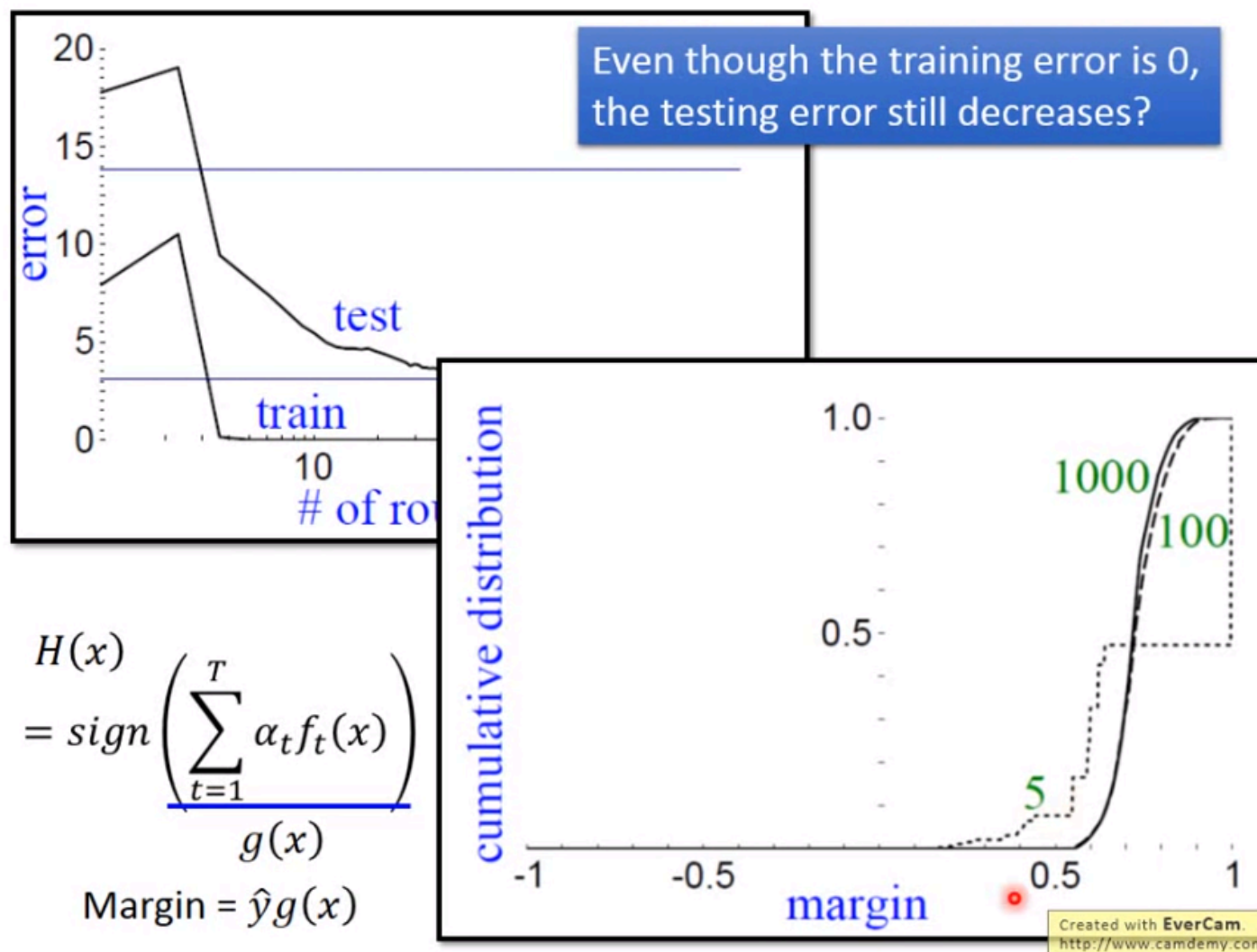
- $Z_{t-1} \times 2\sqrt{\epsilon_t (1 - \epsilon_t)}$
 - ϵ_t 為 Error Rate，並且 Error Rate 一定小於 0.5，最大為 0.5 (可見課程最初說明)

- 根號內所得的最大值就是 0.5，即使乘上 2 還是只有 1

從上面的推導可以知道， Z_t 一定比 Z_{t-1} 還要小，無法想像的話就拿起紙筆簡單賦值計算就可以確認。

因此我們知道，訓練的 Error 是會愈來愈小，即 Z_t 會愈來愈小，所以 Upper Bound 就會愈來愈小。

Adaboost Incredible



上圖可以看到，在訓練資料集上，Adaboost的收斂非常快，大約在 $T=5$ 的地方 Error Rate 就收斂到接近 0_{註1}，但是很神奇的是，即使已經接近於 0，沒有東西可以再學習的情況下還是可以讓測試資料集的 Error Rate 再下降。

說明如下：

- $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t f_t(x))$
 - $\sum_{t=1}^T \alpha_t f_t(x) = g(x)$
- $g(x) \times \hat{y}$ 定義為 Margin
 - 我們希望 $g(x)\hat{y}$ 是同號，同號代表正確，並且希望相乘之後數值愈大愈好
 - $g(x)$ 大於 0 愈多愈好

上圖二可以發現到不同 T 所擁有的 Margin 的差異，即使 T=5 的時候已經讓 Error Rate 不再下降(因為所有 $g(x) \times \hat{y}$ 皆大於 0)，但增加更多 weak classifier 是可以增加 Margin，增加 Margin 的好處在於可以讓結果更 Robust

註1：Adaboost 假設 Error Rate 不會為 0，若為 0 會造成計算 α 錯誤

$H(x)$ ：所有 weak classifier 結合的結果

Large Margin?

Large Margin?

$$H(x) = \text{sign} \left(\underbrace{\sum_{t=1}^T \alpha_t f_t(x)}_{g(x)} \right)$$

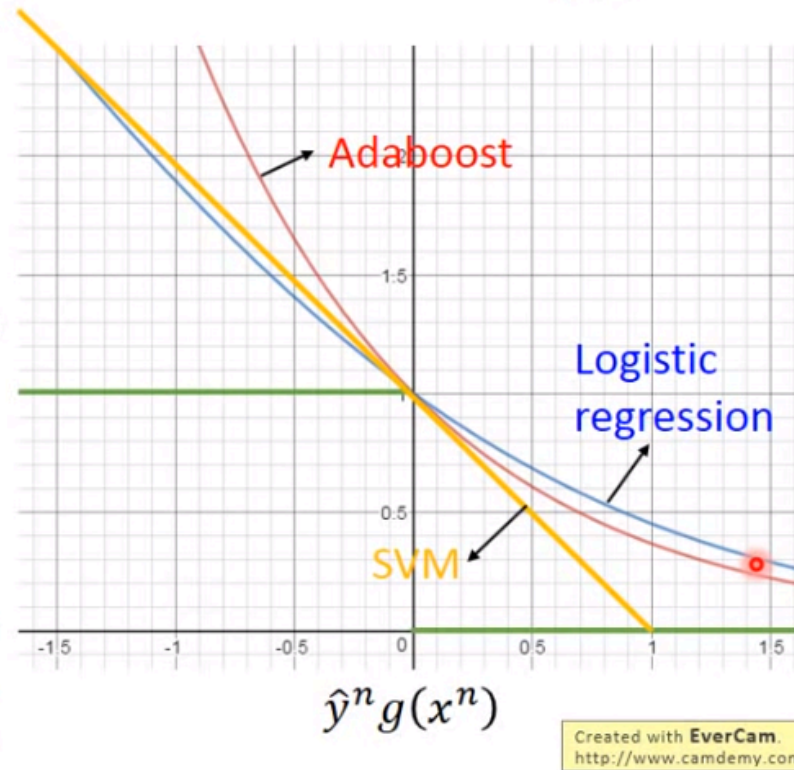
Training Data Error Rate =

$$= \frac{1}{N} \sum_n \delta(H(x^n) \neq \hat{y}^n)$$

$$\leq \frac{1}{N} \sum_n \exp(-\hat{y}^n g(x^n))$$

$$= \prod_{t=1}^T 2\sqrt{\epsilon_t(1-\epsilon_t)}$$

Getting smaller and smaller as T increase



上圖是三種演算法 Margin 差異，剛才的推導我們知道，每一次的迭代都可以讓 Z_t 會愈來愈小，也就是 Upper Bound 會愈來愈小，即使我們根本沒有對 Upper Bound 做微分之類的優化。

直觀來看，只要能讓 $g(x) \times \hat{y}$ 到右側，那 Error Rate 就是 0，但 Adaboost 與 SVM、LR 一樣，即使在右側，它的 Error 也不會是 0，而是可以繼續往下得到更小的 Error，這也是為什麼愈大的 T 可以得到愈好的 Margin 的原因。

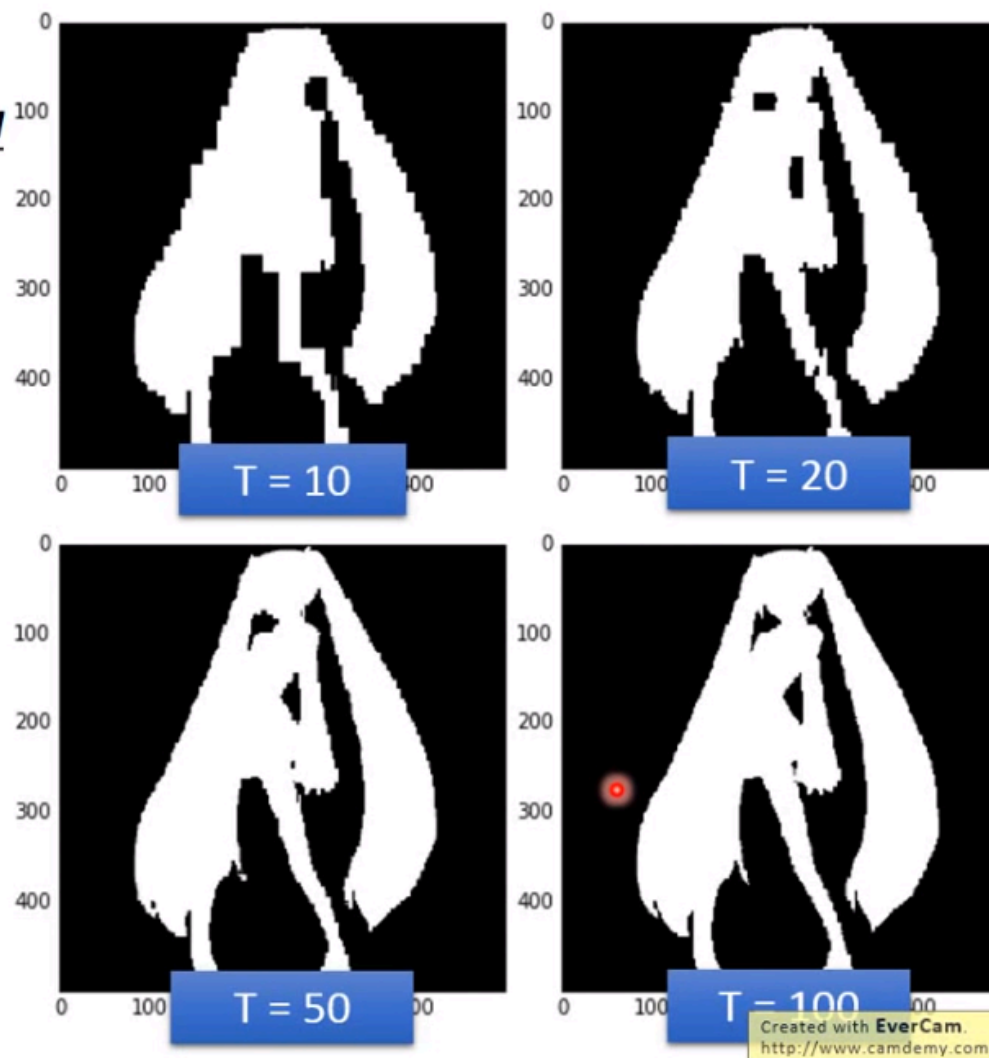
註：可以將 Upper Bound 視為 Adaboost 的 Objective Function(目標函數)

Function of Miku

Experiment: Function of Miku

Adaboost
+Decision Tree

(depth = 5)



上圖是利用 Adaboost+Decision Tree，並且設定深度為 5。

在課程開始的時候也有相同的範例，這種深度根本不可能完成一個初音 Function，但讓人驚豔的是，深度為 5 並且 $T=10$ 的時候(十棵深度為 5 的 Decision Tree)已經可以擁有初音造型，不同於 Random Forest，這十棵樹是互補，在 $T=100$ 的時候已經是完美擬合出初音的造型。

Gradient Boosting

Gradient Boosting

• General Formulation of Boosting

- Initial function $g_0(x) = 0$
- For $t = 1$ to T :
 - Find a function $f_t(x)$ and α_t to improve $g_{t-1}(x)$
 - $g_{t-1}(x) = \sum_{i=1}^{t-1} \alpha_i f_i(x)$
 - $g_t(x) = g_{t-1}(x) + \alpha_t f_t(x)$
- Output: $H(x) = \text{sign}(g_T(x))$

What is the learning target of $g(x)$?

$$\text{Minimize } L(g) = \sum_n l(\hat{y}^n, g(x^n)) = \sum_n \exp(-\hat{y}^n g(x^n))$$

1. 每次迭代都找一個 $f_t(x)$ 與 $\alpha_t(\text{weight})$ 來 improve (提升) $g_{t-1}(x)$
 - $g_{t-1}(x)$ 為過去所有已找出來的 function 做 weight sum 的結果

2. $g_t(x) = g_{t-1}(x) + \alpha_t f_t(x)$

3. Output : $H(x) = \text{sign}(g_T(x))$

 <https://hackmd.io/@shaoeChen/HJ5ldDoSE?type=view> /  李宏毅_ML_Lecture_22 - HackMD  Try  HackMD https://hackmd.io?utm_source=view-page&utm_medium=logo-nav

問題在於如何找到一個 f_t 讓它來提升 g_{t-1} ，因此我們必需設置一個 Objective Function (目標函數) 來優化 (Minimize Cost Function 或 Maximize Objective Function)。

- Cost Function

- $L(g) = \sum_n l(\hat{y}^n, g(x^n))$

- 加總每一筆資料的差異
- 小寫 l 為 loss function
 - 使用 Cross Entropy or Mean Square Error...
 - $\sum_n \exp(-\hat{y}^n g(x^n))$

Gradient Boosting

Gradient Boosting

- Find $g(x)$, minimize $L(g) = \sum_n \exp(-\hat{y}^n g(x^n))$
 - If we already have $g(x) = g_{t-1}(x)$, how to update $g(x)$?

Gradient Descent:

$$g_t \leftarrow g_{t-1} - \eta \left. \frac{\partial L(g)}{\partial g} \right|_{g = g_{t-1}}$$

Same direction

$$\sum_n \exp(-\hat{y}^n g_{t-1}(x^n)) (\hat{y}^n)$$

$$g_t(x) \leftarrow g_{t-1}(x) + \alpha_t f_t(x)$$

Created with EverCam.
http://www.camdemy.com

如何找出一個新的 g_t 來優化 Cost Function(L)?這就需要利用梯度下降來處理，將 g 對 L 做微分計算出 Gradient，再利用這個 Gradient 去更新 g_{t-1} 來得到 g_t (註1)

如何由 Boosting 的角度來看的話，尋找一個 $\alpha_t f_t(x)$ 加到 $g_{t-1}(x)$ 之後變為 $g_t(x)$ ，而其中的 $\alpha_t f_t(x)$ 就是上面所談的偏微分部份 $\eta \frac{\partial L(g)}{\partial g}$ ，只要兩者目標一致，那就一樣可以達到讓 $g_t(x)$ 變小。

註1：雖然 $g(x)$ 是一個 function，但其中每一個點的變化都還是會影響著 L ，因此還是可以對 function 做偏微分。

Gradient Boosting

Gradient Boosting

$$f_t(x) \longleftrightarrow \sum_n \exp(-\hat{y}^n g_t(x^n)) (\hat{y}^n)$$

Same direction

We want to find $f_t(x)$ maximizing

$$\sum_n \frac{\exp(-\hat{y}^n g_{t-1}(x^n)) (\hat{y}^n) f_t(x^n)}{\text{example weight } u_t^n}$$

Same sign

$$u_t^n = \exp(-\hat{y}^n g_{t-1}(x^n)) = \exp\left(-\hat{y}^n \sum_{i=1}^{t-1} \alpha_i f_i(x^n)\right)$$

$$= \prod_{i=1}^{t-1} \exp(-\hat{y}^n \alpha_i f_i(x^n))$$

Exactly the weights we obtain in Adaboost

Created with EverCam.
<http://www.camdemy.com>

我們希望 $f_t(x)$ 的方向與 $\sum_n \exp(-\hat{y}^n g_t(x^n)) (\hat{y}^n)$ (偏微分項) 愈一致愈好 (雖然兩者皆為擁有無窮多維的

Vector，但如果僅考慮訓練資料集，那就是有限維度)。

因此我們希望可以找到一個 $f_t(x)$ 來最大化 $\sum_n \exp(-\hat{y}^n g_{t-1}(x^n)) (\hat{y}^n) f_t(x^n)$ ，值愈大，就代表兩者愈一致，也就是希望 $\hat{y}^n$ 與 $f_t(x)$ 是同號，並且每一筆資料的前面都乘上權重 $\exp(-\hat{y}^n g_{t-1}(x^n))$

將權重 u_t^n 式子推導之後會發現，它就是 Adaboost 的獲得權重的方式。

Gradient Boosting

Gradient Boosting

- Find $g(x)$, minimize $L(g) = \sum_n \exp(-\hat{y}^n g(x^n))$

$$g_t(x) = g_{t-1}(x) + \alpha_t f_t(x)$$

α_t is something like learning rate

Find α_t minimizing $L(g)$

$$\begin{aligned} L(g) &= \sum_n \exp(-\hat{y}^n (g_{t-1}(x^n) + \alpha_t f_t(x^n))) \\ &= \sum_n \exp(-\hat{y}^n g_{t-1}(x^n)) \exp(-\hat{y}^n \alpha_t f_t(x^n)) \\ &= \sum_{\hat{y}^n \neq f_t(x^n)} \exp(-\hat{y}^n g_{t-1}(x^n)) \exp(\alpha_t) \\ &\quad + \sum_{\hat{y}^n = f_t(x^n)} \exp(-\hat{y}^n g_{t-1}(x^n)) \exp(-\alpha_t) \end{aligned}$$

Find α_t such that

$$\frac{\partial L(g)}{\partial \alpha_t} = 0$$

$$\alpha_t = \frac{1}{\ln \sqrt{(1 - \varepsilon_t) / \varepsilon_t}}$$

Created with EverCam.
<http://www.camdemy.com>

現在我們發現到，在 Gradient Boosting 找到的 f_t ，就是 Adaboost 找到的 f_t ，而 α_t 的作用就跟 Learning rate 一樣，只是我們要窮舉一切可能的值來試如何讓 $g_t(x)$ 的 L 最小。

這麼做的理由在於，光是找出 f_t 就可能耗盡很大的資源，與 NN 不同，NN 即使你的學習效率過低，只要多幾次的迭代還是可以收斂，

因此在 Gradient Boosting 的部份會固定住 f_t 然後窮舉一切可能尋找 α_t 來讓 $g_t(x)$ 的 L 掉最多。

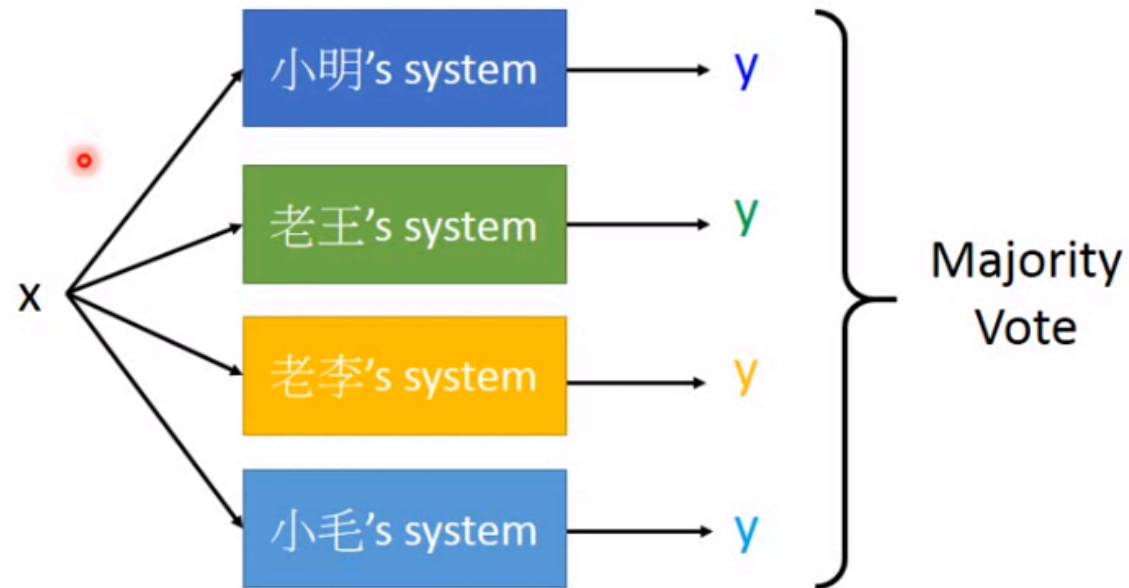
實際做法就是找一個 α_t ，它對 L 的偏微分為 $0(\frac{\partial L(g)}{\partial \alpha_t})$ ，那就是一個極值。

更巧合的是，找出來 α_t 就是 $\ln \sqrt{(1 - \epsilon_t) / \epsilon_t}$ ，就是 Adaboost 裡面的 Weight。

很酷的範例：http://arogozhnikov.github.io/2016/07/05/gradient_boosting_playground.html
(http://arogozhnikov.github.io/2016/07/05/gradient_boosting_playground.html)

Voting

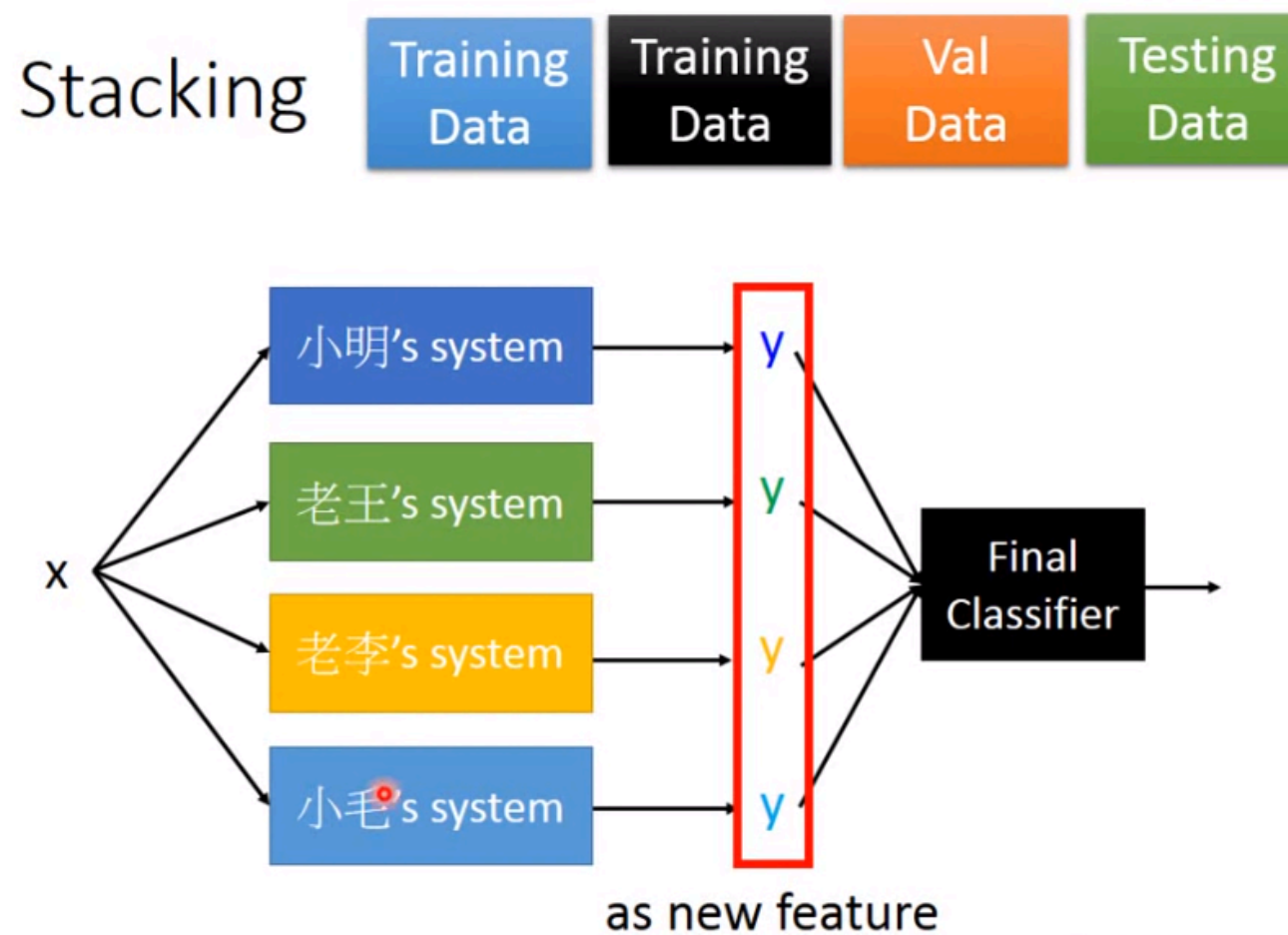
Voting



Created with EverCam.
<http://www.camdemy.com>

直觀來看，你有多個 Model，一筆資料進去之後各自擁有 Outout，再結合四個答案來決定一個答案。如果是分類，那就可以用 Majority Vote，多個模型最多 Output 的類別來決定你的最終類別。

Stacking



Created with EverCam.
<http://www.camdemy.com>

但如果多個模型中好壞不一定的時候，以 Voting 的方式可能會得到不好的結果，這時候可以在多個模型的多出後面再加上一個最終的分類器，將多模型的輸出當做輸入，來做最後的決定。

如果前面已經是很複雜的模型，那最後可能只可以使用 Logistic Regression 就可以。

要注意的是，採用這個方式要將原本的訓練資料集再拆成兩份，一份訓練前面多組模型，一份訓練最終的輸出分類器。